

Understanding Java File Operations

INPUT/OUTPUT AND STREAMS IN JAVA



Alex Liu

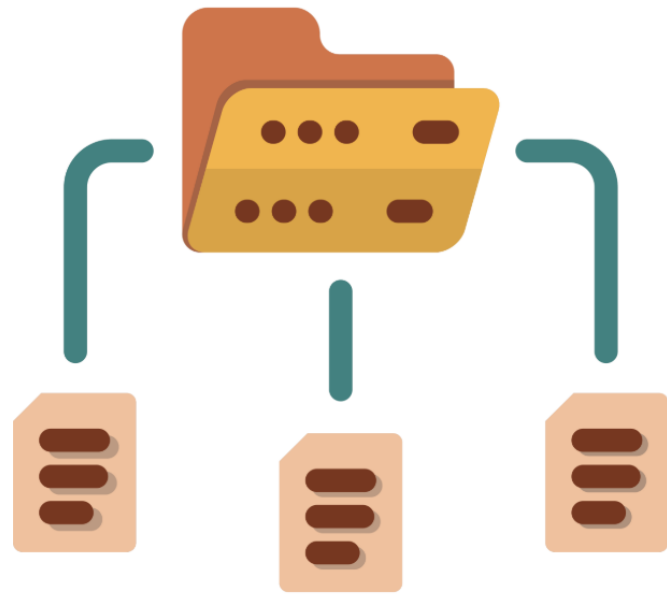
Software Development Engineer

About Alex Liu

- M.S. Degree in Computer Science
- 8 years of experience in the software development industry
- 6 years of hands-on expertise in Java programming



Course overview



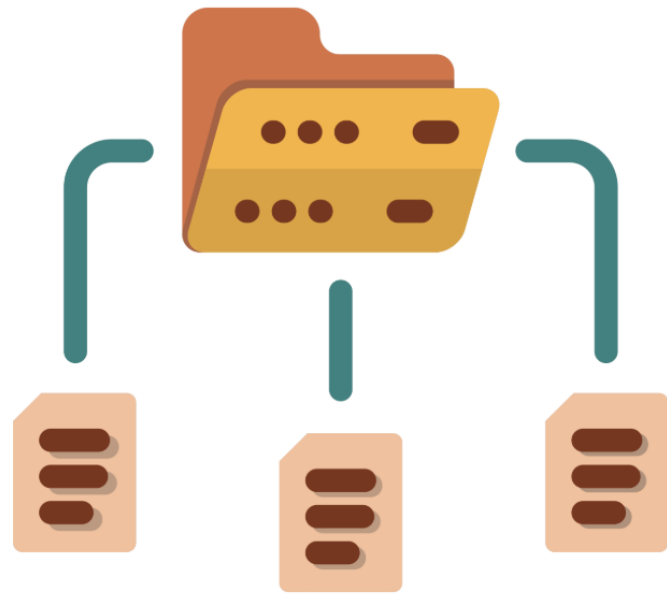
Java File Operations

Creation

Deletion

Directory management

Course overview



Java File Operations

Creation

Deletion

Directory management

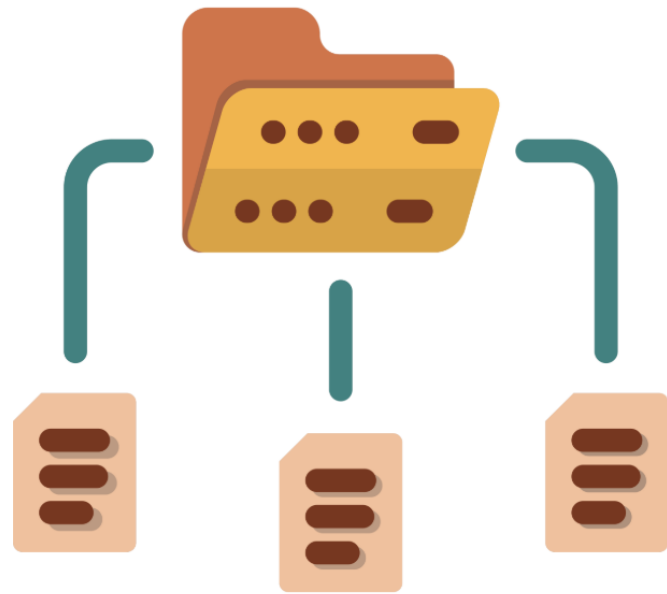


Iterators and Streams

Processing collections

Transforming data

Course overview



Java File Operations

Creation

Deletion

Directory management



Iterators and Streams

Processing collections

Transforming data



Custom Methods and More

Build scalable

Java applications

Creating files

- Import the `File` class

```
import java.io.File;
```

- Make a `File` object named `dataTextFile`

```
File dataTextFile = new File("data.txt");
```

- To create the file on the computer, call the `.createNewFile()` method

```
boolean result = dataTextFile.createNewFile();
```

- Returns `true` if the file is created successfully; `false` if it already exists
- Works for any file type

Deleting files

- Create a `File` object named `exampleFile` that references the file we want to remove

```
File exampleFile = new File("example.txt");
```

- Use the `.delete()` method to attempt to delete the file

```
boolean deleted = exampleFile.delete();
```

- Returns `true` if the file was deleted
- Returns `false` if file could not be deleted (e.g., due to insufficient permissions)

Checking file existence

- The `.exists()` method returns `true` if the file exists

```
// Use `.exists()` to check if a file already exists on our disk
if (dataTextFile.exists()) {
    // Print message if file exists
    System.out.println("The file already exists.");
} else {
    // Attempt to create the file if it doesn't exist
    boolean result = dataTextFile.createNewFile();
}
```

- Helps prevent duplicates

Wrapping file operations with try-catch

- Always wrap file operations in a try-catch block
 - Handles potential exceptions like `IOException`
 - From permission errors or invalid files

```
try {  
    //Any file operation  
} catch (IOException e) {  
    e.getMessage();  
}
```

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA

Reading and writing files

INPUT/OUTPUT AND STREAMS IN JAVA



Alex Liu

Software Development Engineer

Casting

- Converting between different primitive data types
- Widening casting - automatically for smaller -> larger size
 - `byte` -> `short` -> `char` -> `int` -> `long` -> `float` -> `double`

```
// Widening casting - happens automatically
```

```
byte myByte = 8;  
int myInt = myByte;
```

- Narrowing casting - manual, larger -> smaller

```
// Narrowing casting - is manual
```

```
double myDouble = 3.14;  
int myInt = (int) myDouble; // Note the (int)
```

Understanding specific casting for reading file

- Casting from `int` to `char`: `char data = (char) 123;`

```
// Unicode value for character 'a'
int data = 97;
// Directly print the data in `int` format
System.out.print(data);
// Cast data from `int` to `char` and print
System.out.print((char)data);
```

97

a

¹ <https://www.ascii-code.com/>

Reading data from a file using FileReader

- Import the `FileReader` class to enable file reading functionality

```
import java.io.FileReader;
```

- Read the file, print character by character and then close the file

```
FileReader fr = new FileReader("example.txt");  
int data = fr.read();  
// Read and print each character  
while (data != -1) {  
    System.out.print((char) data);  
    data = fr.read();  
}  
fr.close(); // Close the file to release resources
```

Efficient reading with BufferedReader

- `BufferedReader` - `import java.io.BufferedReader;`
 - Reads text files line by line
 - More efficient for larger files

```
// Create a BufferedReader object
BufferedReader br = new BufferedReader(new FileReader("example.txt"));
String line;
// User .readLine() to read the file line by line
while ((line = br.readLine()) != null){
    System.out.println(line);
}
br.close();
```


Writing data using FileWriter

- `FileWriter` :
 - **Overwrites** existing content by default - use caution to prevent data loss!
- Import `FileWriter`

```
import java.io.FileWriter;

// Create a new `FileWriter` Object
FileWriter fw = new FileWriter("example.txt");
// Write the data to file using `.write()`
fw.write("Overwriting the file.");
fw.close()
```


Appending data using FileWriter

- `FileWriter`
 - Allows appending data to files by setting to `append` mode without overwriting existing content

```
// Initialize `FileWriter` with append mode
FileWriter fw = new FileWriter("example.txt", true);
// Append data to the file
fw.write("Appending data.");
fw.close()
```

Efficient writing with BufferedWriter

- `BufferedWriter` - `import java.io.BufferedWriter;`
 - Writes text more efficiently using larger chunks of text
 - Minimizes write operations

```
// Create a BufferedWriter object
BufferedWriter bw = new BufferedWriter(new FileWriter("example.txt"));
bw.write("Writing data");

// .newLine() to add line breaks
bw.newLine();
bw.close();
```

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA

Managing directories and paths

INPUT/OUTPUT AND STREAMS IN JAVA



Alex Liu

Software Developer Engineer

Key concept

- Directories organize files and data
- Paths specify the location of files and directories
- **File** class can represent either a file or a directory
 - The actual resource created depends on the method called (`mkdir()` for directories and `createNewFile()` for files)



Creating directories

- Import the `File` class

```
import java.io.File;
```

- Creates a `File` object representing the directory

```
File newDirectory = new File("myDirectory");
```

- Use `.mkdir()` creates the actual directory on the file system

```
boolean created = newDirectory.mkdir();
```

- Returns `true` if the directory is created successfully
- Returns `false` if the directory already exists or cannot be created

Listing files in a directory

- Use `.listFiles()` to list files in directory

```
// Creates a `File` object representing the directory `myDirectory`  
File dir = new File("myDirectory");  
//Retrieves an `array` of `File` objects representing the contents of `myDirectory`  
File[] files = dir.listFiles();
```

- Returns an `array` of `File` objects if the directory exists
- Returns `null` if the directory does not exist

Retrieving a file's relative Path

- **Relative Path:** location of a file/directory in relation to the current working directory

```
// Creates a `File` object for `sample.txt` inside `myDirectory`  
File file = new File("myDirectory/sample.txt");  
// Retrieves the relative path of the file as a String  
String path = file.getPath();  
System.out.println(path);
```

- We assume `myDirectory` already exists; otherwise, file creation or writing will fail
- Output:

```
myDirectory/sample.txt
```


Working with absolute paths

- **Absolute Path:** the full path from the root of the file system

```
// Creates a `File` object for `sample.txt` inside `myDirectory`  
File file = new File("myDirectory/sample.txt");  
// Retrieves the absolute path of the file as a String  
String absPath = file.getAbsolutePath();  
System.out.println(absPath);
```

- Output:

```
/user/home/myDirectory/sample.txt
```

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA